In addition to server validation, ensure that the UI also limits user input for the prompt to a maximum of 1,000 characters. This provides immediate feedback to the user and prevents them from submitting prompts that exceed the limit:

```
<textarea maxLength={1000} ... />
```

Of course, input validation is only a first step in securing your web application, as it must be part of a broader security strategy. Establishing reasonable defaults for input validation helps mitigate common vulnerabilities, but additional layers of security controls are essential.

9.1.2 Security middleware layer

The security middleware layer acts as a pivotal decision point in your application, analyzing incoming requests to determine their safety. This intermediate layer functions as a security system that employs multiple security controls and algorithms to detect potential threats. The middleware performs several key functions:

- *Analyzes request patterns*—It examines the frequency and nature of incoming requests to identify any unusual patterns that may indicate malicious activity, such as brute-force attacks or denial-of-service attempts.
- *Checks for known attack signatures*—The middleware utilizes a database of known attack signatures (such as SQL injection patterns or cross-site scripting attempts) to flag requests that match these signatures.
- *Validates authentication tokens*—If you employ authorization or authentication frameworks, it verifies the authenticity of tokens (like JWTs) included in requests to ensure that users are who they claim to be and have the necessary permissions.
- *Examines request headers and metadata*—The middleware inspects headers and metadata for anomalies, such as unusual user agents or IP addresses, which might suggest a spoofing attempt.
- *Applies machine learning models for threat detection*—By using machine learning algorithms, the middleware can identify emerging threats based on historical data and adapt to new attack vectors over time.
- *Rate limits requests*—Stops excessive requests that can flood your backend services.

The middleware ultimately makes a binary decision, either flagging the input as malicious or marking it as safe for further processing. It should be part of your overall security model and be scalable enough to handle the incoming load of requests from your clients.

> **NOTE** For optimal effectiveness, the security middleware layer should be positioned at the very beginning of your request-processing pipeline. This placement ensures that all incoming requests are intercepted and analyzed before they reach any part of your application's core logic. When deploying Next.js applications on platforms like Vercel, for example, these middleware functions